

FB15k-237 Link Prediction: Baseline Training and Preliminary Experiments

Thalia Delhaise (66937), Lenny Briclet (66926), André Fonseca (58225), Rafael Gufler (66081)

Project 425282 - Report for Milestone 1 (Deep Learning)

Abstract

We build the multi-relational **FB15k-237** graph, train **TransE** and **RotatE** under **matched** settings and **filtered** ranking, and report preliminary **MRR** / **Hits@k** and training curves. Next work centers on **custom hard-negative mining** (PyTorch) and **relation-wise** / error analysis; a **GNN** is **optional** for the final project if time allows. Code: `code/train_baseline_kge.py`. See Milestone 0 topic proposal: `milestone0_dl/topic_proposal.tex`.

1. Introduction and plan

Goal: given training triples (h, r, t) in a knowledge graph, rank the correct missing entity for queries $(h, r, ?)$ and $(?, r, t)$. We compare **TransE** to the stronger **ROTATE** [4] on **identical** train/val/test partitions and the same **filtered** evaluation (true triples in any split are not treated as false tails/heads) per Bordes et al. [2]. The question is: *where* **ROTATE** gains over **TransE** (aggregate vs. relation/degree slices), and later whether **custom** hard negatives change that picture vs. default sampling. Milestone 1: **data ready**, **baseline** end-to-end, **preliminary** metrics/curves, and a **plan** for hard negatives + slices (GNN only if time).

2. Problem statement, dataset, and evaluation

Dataset. FB15k-237: 14,541 entities, 237 relations, 272,115 / 17,535 / 20,466 train, validation, and test triples (official split) [4]. The graph is $\mathcal{G} = (\mathcal{E}, \mathcal{R}, \mathcal{T})$, $\mathcal{T} \subseteq \mathcal{E} \times \mathcal{R} \times \mathcal{E}$. No extra subsampling.

Metrics. Filtered mean reciprocal rank (MRR) and Hits@ k ($k \in \{1, 3, 10\}$) on head and tail prediction, then averaged, as in standard benchmarks.

Expected. A controlled comparison under matched hyperparameters: same embedding size d , same optimizer family, same maximum epochs (with early stopping on val MRR), and same number of corruptions per positive where applicable.

3. Data pipeline and preprocessing

Download. The benchmark is loaded via **PyKEEN** [1] (dataset name `fb15k237`, cached locally on first use). The library maps each entity and relation to integer IDs and materializes a **TriplesFactory** for train, validation, and test.

Check. We verify triple counts and disjointness of test from train. For Milestone 1, we will add simple **statistics** (to appear in the next version / Milestone 2): histogram of **relation** frequency and of entity **in-degree** / out-degree, plus one example each of a roughly one-to-many and many-to-one pattern for qualitative discussion.

4. Baseline models and training setup (PyKEEN)

We start with two classical models with fixed scoring: **TransE** and **ROTATE** [4]. The rest of this section is the **configuration we actually run** (edit `train_baseline_kge.py` if you change it).

Model	TransE, RotatE (separate runs)
Formulation	Entity embeddings in $\mathbb{R}^d$; score triples; rank tails/heads.
Input / shape	Mini-batch of triple indices; shape <code>[batch, 3]</code> .
Negatives	Corrupt head or tail; Bernoulli as in KGE default practice.
Loss	Margin / pairwise ranking (default per model in PyKEEN).
Optimiser	Adam or AdamW; small weight decay.
Hyperparam.	$d = 128$; up to 50 epochs; <code>batch_size=512</code> ; LR 10^{-3} ; early stop if val MRR flat 10 epochs.
Seed / env.	<code>random_seed=42</code> ; Python 3, PyTorch, PyKEEN; GPU optional.
Time budget	(fill wall-clock after run)

The script writes `artifacts/baseline/<Model>_summary.txt` and a PyKEEN result folder; copy key metrics and loss curves from the run into the table and figure above.



Figure 1: Baseline KGE data flow for Milestone 1. Later: custom hard-negative training and slice evaluation; GNN (optional).

5. Preliminary results and training curves

Fill in after you run the baseline script. The table and figure below are **placeholders**; replace with real numbers and plots (PNG/PDF) exported from the training log or TensorBoard/CSV.

Table 1: Preliminary filtered metrics on the test set (replace with your run).

Model	MRR	H@1	H@3	H@10	Notes
TransE	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	same d , budget
RotatE	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	idem



Insert training loss and validation MRR vs. epoch (e.g. from PyKEEN log or CSV).

Figure 2: Placeholder for training/validation curves.

6. Known issues and next steps

Current issues. (1) Hyperparameters may need tuning. (2) Default negative sampling: **custom** hard-negative mining (PyTorch) + ablations are the **next** main deliverable after baselines. (3) GNN (e.g. R-GCN [3]): **optional** stretch goal.

Milestone 2+ plan. (a) Custom negatives vs. default. (b) Relation- and degree-**slices** + short **error** study. (c) Optional: d or epoch budget. (d) Optional GNN on same split. (e) Compute vs. accuracy. One script per experiment; optional notebook for plots.

Milestone 1 deliverables checklist. ✓ Data available via PyKEEN; ✓ ID mappings; ✓ baseline **model + architecture + loss + optim + budget** documented; ✓ skeleton + runnable training script; ✓ preliminary **results** (fill TBD) and curve placeholders; ✓ this report; ☐ **fill in** actual metrics and GPU time before upload.

Code repository. Baseline training: `code/train_baseline_kge.py`. Unify with KG course: same dataset and protocol.

References

- [1] Mehdi Ali, Max Berrendorf, Charles Tapley Hoyt, Laurent Vermue, et al. PyKEEN 1.0: A python library for training and evaluating knowledge graph embeddings. *JMLR*, 22(82):1–6, 2021.
- [2] Antoine Bordes, Nicolas Usunier, Alberto Garcia-Durán, Jason Weston, and Oksana Yakhnenko. Translating embeddings for modeling multi-relational data. In *NeurIPS*, 2013.

- [3] Michael Schlichtkrull, Thomas N Kipf, et al. Modeling relational data with graph convolutional networks. In *ESWC*, 2018.
- [4] Zhiqing Sun, Zhi-Hong Deng, Jian-Yun Nie, and Jian Tang. RotatE: Knowledge graph embedding by relational rotation in complex space. In *ICLR*, 2019.